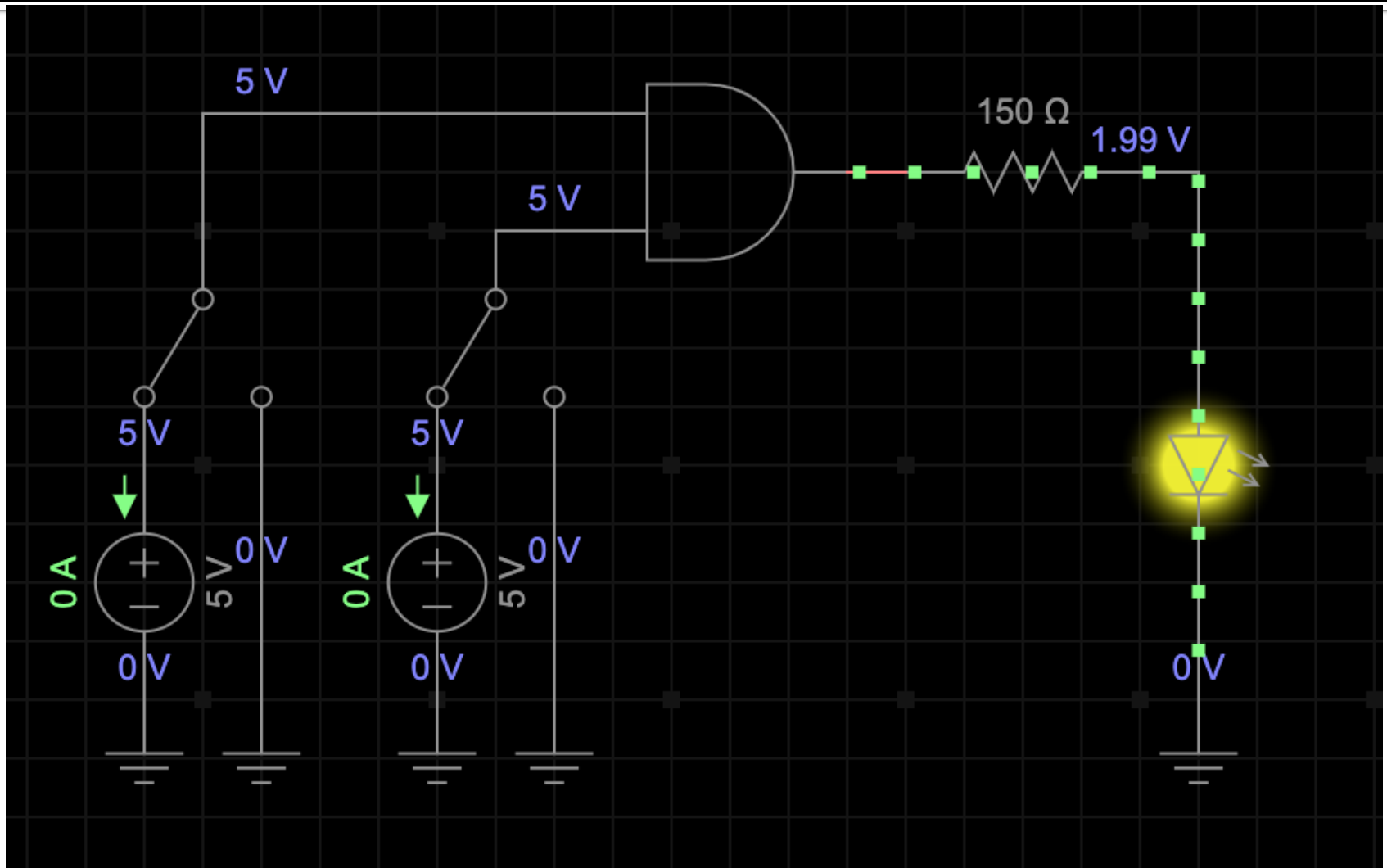


Introduction to Boolean Algebra and
Logic Circuits

**CSCI 1101 Introduction to
Computer Science**

Do you recognize this?

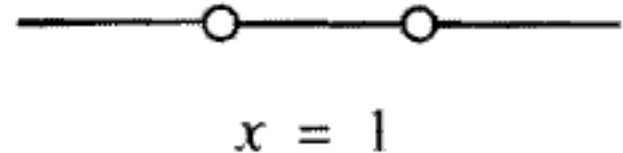
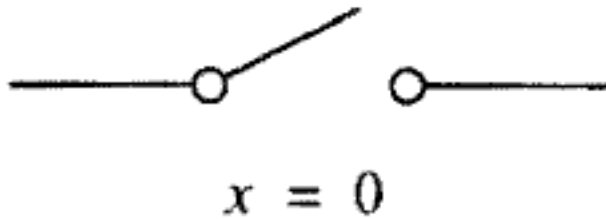


Circuits and Boolean Algebra

- Computers are constructed from logic circuits
 - Logic circuits perform operations on digital signals
 - Usually implemented as circuits where signal values are restricted to a few values (e.g., binary circuits)

Binary Circuits

- Circuit with 2 possible states, open ($x = 0$) or closed ($x = 1$), where x is the input variable that controls the switch

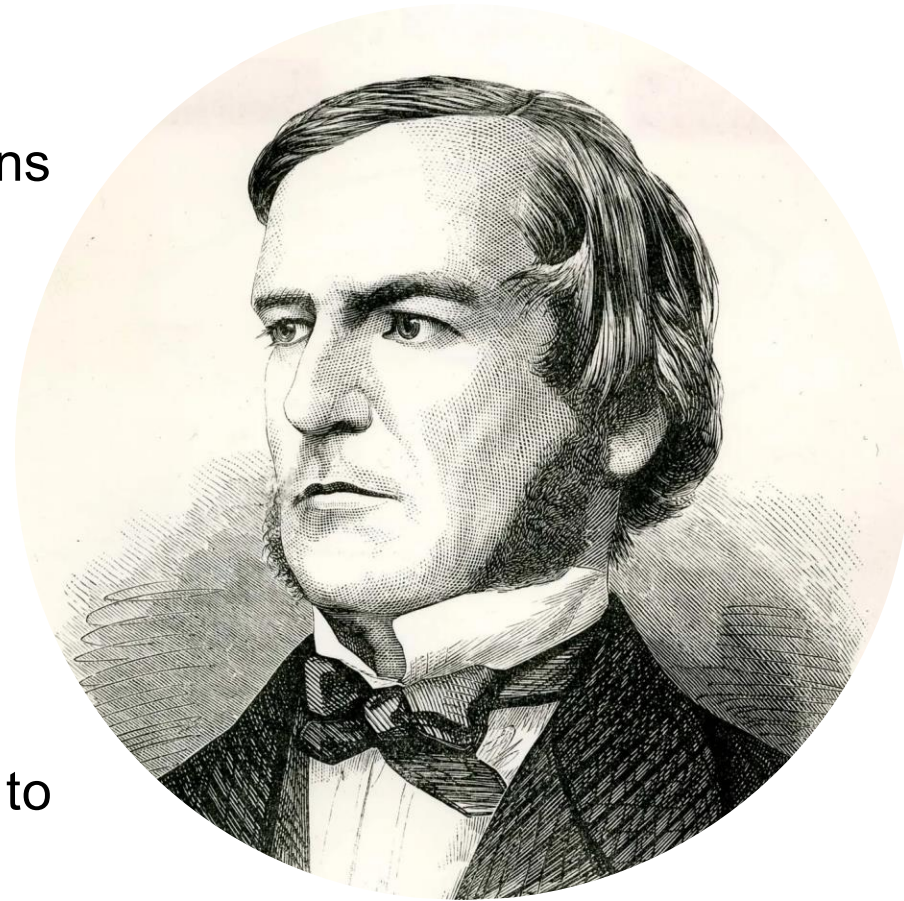


- Now, a short intro to **Boolean algebra**

George Boole

Computer science pioneers

- **George Boole**, a mathematician and logician, made significant contributions to computer science.
- **Boolean algebra**, a mathematical scheme for describing in algebra the processes involved in logical reasoning (1849)
- In 1930's, **Claude Shannon** showed that Boolean algebra can be applied to electrical circuits by using **switches**.



Boolean Algebra

- **Boolean algebra** is a branch of mathematics that deals with variables that have two possible values: **true** and **false** (often represented as 1 and 0).
- It is foundational to digital logic and computer science, especially in designing circuits and algorithms.
- The **basic operations** of Boolean algebra are
 - **AND** (Conjunction)
 - **OR** (Disjunction)
 - **NOT** (Negation)

AND Operations

- An **AND** function of variables $x_1, x_2, \dots, x_n$ has the value 1 only if all n variables are equal to 1
- AND takes in two or more variables/inputs
- AND is like multiplication

OR Operations

- An OR function of variables $x_1, x_2, \dots, x_n$ has the value 1 if at least one, or more, of the n variables are equal to 1
- OR takes two or more variables/inputs
- OR is like addition

NOT Operation

- NOT takes one variable/input
- NOT is true if the input variable is false
 - $\text{NOT}(\text{true}) = (\text{true})' = \text{false}$
 - $\text{NOT}(\text{false}) = \text{true}$

Truth Table

- A **truth table** is a structured representation that presents all possible combinations of truth values for the input variables of a Boolean function

Truth Table (Example)

- Used to define logic operations

x_1	x_2	$x_1 \odot x_2$	$x_1 \oplus x_2$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Truth table for AND and OR operations with 2 inputs, x_1 and x_2

AND

OR

Truth Tables Multiple Inputs

- AND and OR can be extended to n variables

x_1	x_2	x_3	$x_1 \cdot x_2 \cdot x_3$	$x_1 + x_2 + x_3$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Truth table for AND and OR operations with 3 inputs

What would the truth table for NOT look like?

Axioms of Boolean Algebra

- Boolean algebra is based on set of rules (or basic principles) that define the structure and behavior of Boolean systems
 - **axioms**
- These axioms are the building blocks for developing more complex theorems and operations in Boolean algebra.

Axioms of Boolean Algebra

$$1a. \quad 0 \cdot 0 = 0$$

$$1b. \quad 1 + 1 = 1$$

$$2a. \quad 1 \cdot 1 = 1$$

$$2b. \quad 0 + 0 = 0$$

$$3a. \quad 0 \cdot 1 = 1 \cdot 0 = 0$$

$$3b. \quad 1 + 0 = 0 + 1 = 1$$

$$4a. \quad \text{If } x = 0, \text{ then } \bar{x} = 1$$

$$4b. \quad \text{If } x = 1, \text{ then } \bar{x} = 0$$

Single Variable Rules (Theorems)

- From the axioms, we define rules (**theorems**) for dealing with single variables
- We can prove by **perfect induction** (substitute $x = 0$ and $x = 1$ in each expression and use axioms)

$$5a. \quad x \cdot 0 = 0$$

$$5b. \quad x + 1 = 1$$

$$6a. \quad x \cdot 1 = x$$

$$6b. \quad x + 0 = x$$

$$7a. \quad x \cdot x = x$$

$$7b. \quad x + x = x$$

$$8a. \quad x \cdot \bar{x} = 0$$

$$8b. \quad x + \bar{x} = 1$$

$$9. \quad \bar{\bar{x}} = x$$

Principle of Duality in Boolean Algebra

- Given a logic expression, we get its *dual* by replacing all $+$ with $\cdot$ (and *vice versa*) and by replacing 0s with 1s and 1s with 0s.
- The dual of any true statement, axiom or theorem, in Boolean algebra is also a true statement
- This means that at least 2 different ways exist to express every logic function

Two- and Three- Variable Properties

- We often need to deal with a number of variables
- Define some 2- and 3-variable identities
- Give the dual of each identity
- Often referred to as *properties*
- If x , y , and z are variables in B , the following properties hold...

Properties

10a. $x \cdot y = y \cdot x$

Commutative

10b. $x + y = y + x$

11a. $x \cdot (y \cdot z) = (x \cdot y) \cdot z$

Associative

11b. $x + (y + z) = (x + y) + z$

12a. $x \cdot (y + z) = x \cdot y + x \cdot z$

Distributive

12b. $x + y \cdot z = (x + y) \cdot (x + z)$

13a. $x + x \cdot y = x$

Absorption

13b. $x \cdot (x + y) = x$

14a. $x \cdot y + x \cdot \bar{y} = x$

Combining

14b. $(x + y) \cdot (x + \bar{y}) = x$

Properties

$$15a. \quad \overline{x \cdot y} = \bar{x} + \bar{y}$$

DeMorgan's theorem

$$15b. \quad \overline{\bar{x} + \bar{y}} = x \cdot y$$

$$16a. \quad x + \bar{x} \cdot y = x + y$$

$$16b. \quad x \cdot (\bar{x} + y) = x \cdot y$$

$$17a. \quad x \cdot y + y \cdot z + \bar{x} \cdot z = x \cdot y + \bar{x} \cdot z$$

Consensus

$$17b. \quad (x + y) \cdot (y + z) \cdot (\bar{x} + z) = (x + y) \cdot (\bar{x} + z)$$

Properties

- We can prove by perfect induction (truth tables), algebraic manipulation, or Venn diagrams
- Example: proving DeMorgan's theorem using truth table

x	y	$x \cdot y$	$\overline{x \cdot y}$	$\bar{x}$	$\bar{y}$	$\bar{x} + \bar{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

$\underbrace{\hspace{10em}}$
LHS

$\underbrace{\hspace{10em}}$
RHS

Venn Diagrams

- Using truth tables to verify theorems and properties can be tedious, and may not be conceptually informative
- **Venn diagrams** are tools that provides for more intuitive understanding of how expressions may be equivalent
 - Graphical illustration of set algebra
 - We're not going to talk about them.

Equivalent Notations

The following notations are equivalent:

X' same as $\bar{X}$

$(X + Y)'$ same as $\overline{X + Y}$

Other Notation

- Using the same symbols for arithmetic and logic can be confusing
- We can use the following notation:
 - AND $\wedge$, e.g., $x_1 \wedge x_2$
 - OR $\vee$, e.g., $x_1 \vee x_2$

Precedence of Operations

- In the absence of parentheses, a logic expression is evaluated in the following order:
 1. NOT
 2. AND
 3. OR

Examples

- Fill in the missing output values

X	Y	Z	$X \cdot Y + Z'$
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Examples

- Fill in the missing output values

Solution:

1. No parenthesis.
Simply follow the rules of precedence (slide 24).

X	Y	Z	$X \cdot Y$	Z'	$X \cdot Y + Z'$
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

Examples

- Fill in the missing output values

Solution:

1. No parenthesis.
Simply follow the rules of precedence (slide 24).

X	Y	Z	$X \cdot Y$	Z'	$X \cdot Y + Z'$
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Examples

- Fill in the missing output values

X	Y	Z	$(X + Y) \cdot Z$
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Examples

- Fill in the missing output values

Solution:

1. Parenthesis first
2. Then, follow the rules of precedence

X	Y	Z	$(X + Y)$	$(X + Y) \cdot Z$
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Examples

- Fill in the missing output values

Solution:

1. Parenthesis first
2. Then, follow the rules of precedence

X	Y	Z	$(X + Y)$	$(X + Y) \cdot Z$
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	1
1	0	0	1	0
1	0	1	1	1
1	1	0	1	0
1	1	1	1	1

Computers and Electricity

Gate

A device that performs a basic operation on electrical signals

Circuits

Gates combined to perform more complicated tasks

Gates

Six types of gates

- NOT
- AND
- OR
- XOR
- NAND
- NOR

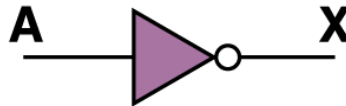
NOT Gate

A **NOT** gate accepts one input signal (0 or 1) and returns the opposite signal as output

Boolean Expression

$$X = A'$$

Logic Diagram Symbol



Truth Table

A	X
0	1
1	0

AND Gate

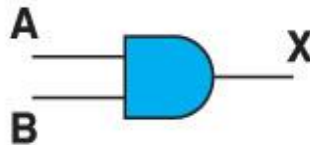
An **AND** gate accepts two input signals

If both are 1, the output is 1; otherwise, the output is 0

Boolean Expression

$$X = A \cdot B$$

Logic Diagram Symbol

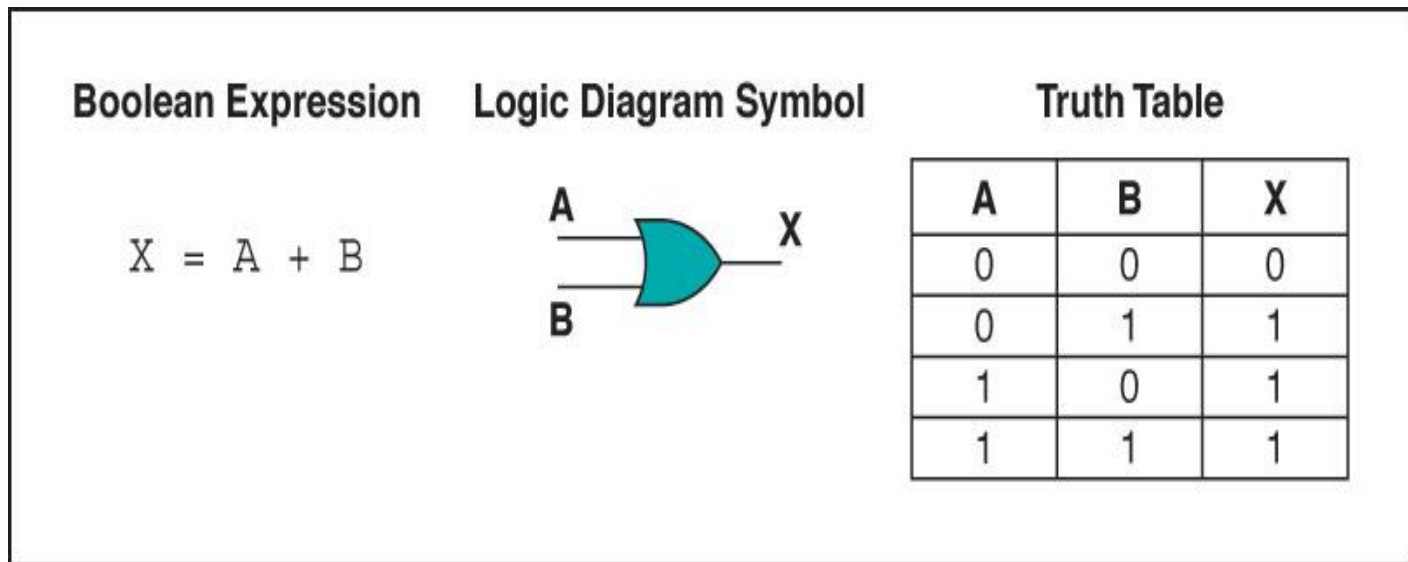


Truth Table

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

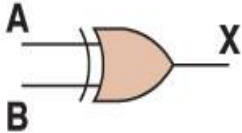
OR Gate

An **OR** (inclusive OR) gate accepts two input signals. If both are 0, the output is 0; otherwise, the output is 1.



XOR Gate

An **XOR** (**exclusive OR**) gate accepts two input signals. If both are the same, the output is 0; otherwise, the output is 1.

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = A \oplus B$		<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	0
A	B	X															
0	0	0															
0	1	1															
1	0	1															
1	1	0															

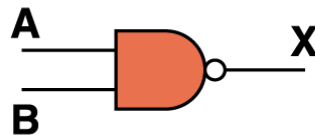
NAND Gate

The NAND gate accepts two input signals
If both are 1, the output is 0; otherwise, the output is 1

Boolean Expression

$$X = (A \cdot B)'$$

Logic Diagram Symbol

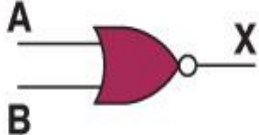


Truth Table

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

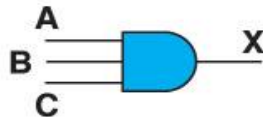
NOR Gate

The NOR gate accepts two input signals
If both are 0, the output is 1; otherwise, the output is 0

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = (A + B)'$		<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	0
A	B	X															
0	0	1															
0	1	0															
1	0	0															
1	1	0															

Gates with More Inputs

- Gates can be designed to accept three or more input values
- A three-input **AND** gate, for example, produces an output of **1** only if all input values are **1**

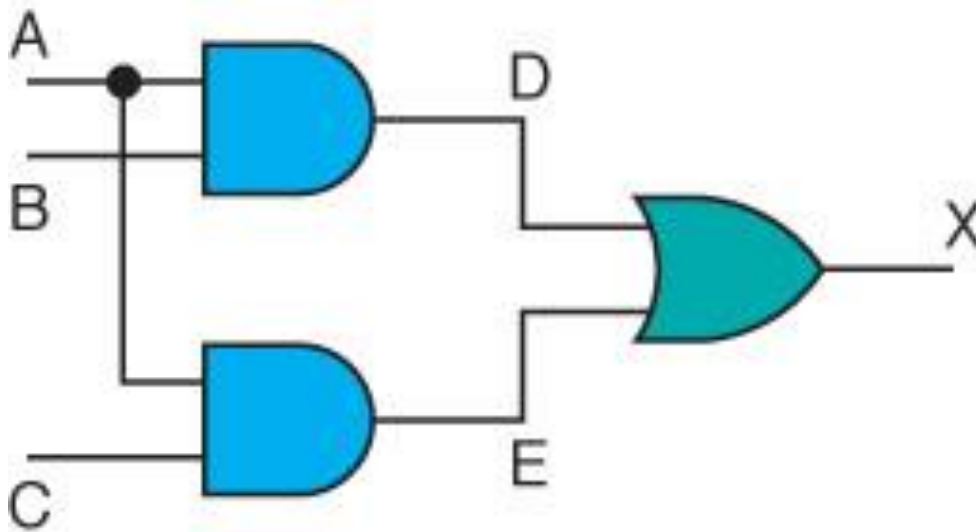
Boolean Expression	Logic Diagram Symbol	Truth Table																																				
$X = A \cdot B \cdot C$		<table><tr><th>A</th><th>B</th><th>C</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	A	B	C	X	0	0	0	0	0	0	1	0	0	1	0	0	0	1	1	0	1	0	0	0	1	0	1	0	1	1	0	0	1	1	1	1
A	B	C	X																																			
0	0	0	0																																			
0	0	1	0																																			
0	1	0	0																																			
0	1	1	0																																			
1	0	0	0																																			
1	0	1	0																																			
1	1	0	0																																			
1	1	1	1																																			

Circuits

- **Combinational circuit**
 - The input values explicitly determine the output
- **Sequential circuit**
 - The output is a function of the input values and the existing state of the circuit
 - We describe the circuit operations using
 - Boolean expressions
 - Logic diagrams
 - Truth tables

Combinational Circuits

Gates are combined into circuits by using the output of one gate as the input for another



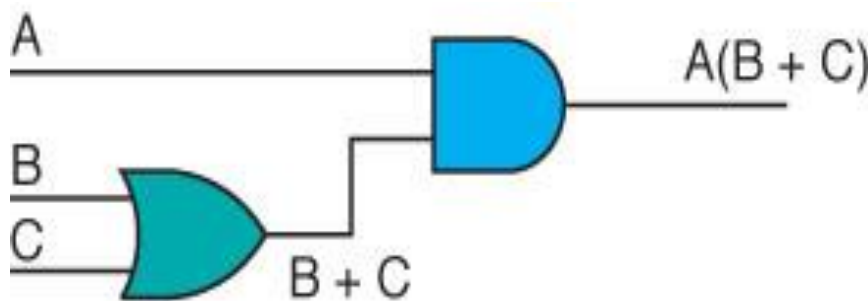
Combinational Circuits

A	B	C	D	E	X
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

- Three inputs require eight rows to describe all possible input combinations
- This circuit is described by the Boolean expression:
 $(AB + AC)$

Combinational Circuits

Consider the following Boolean expression $A(B + C)$



A	B	C	B + C	A(B + C)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

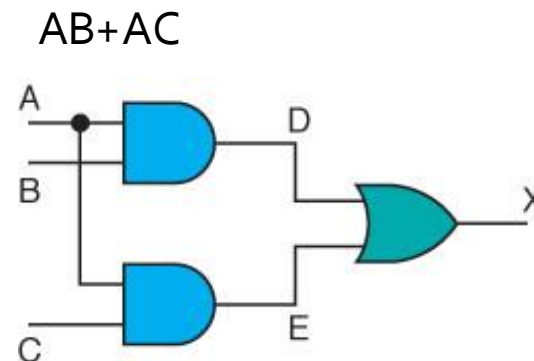
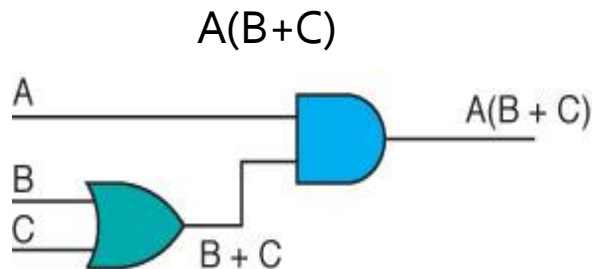
Does this truth table look familiar?

Compare it with previous table...

Combinational Circuits

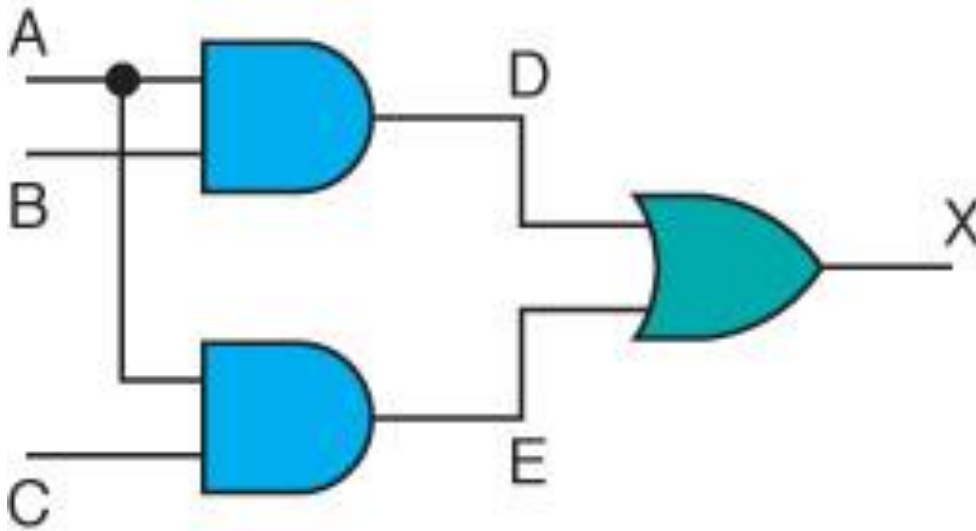
Circuit equivalence

- Two circuits that produce the same output for identical input
- Boolean algebra allows us to apply provable mathematical principles to help design circuits
- $A(B + C) = AB + AC$ (distributive law) so circuits must be equivalent



Example 1

- If $A = 0$, $B = 1$ and $C = 1$, what is x ?



Example 2

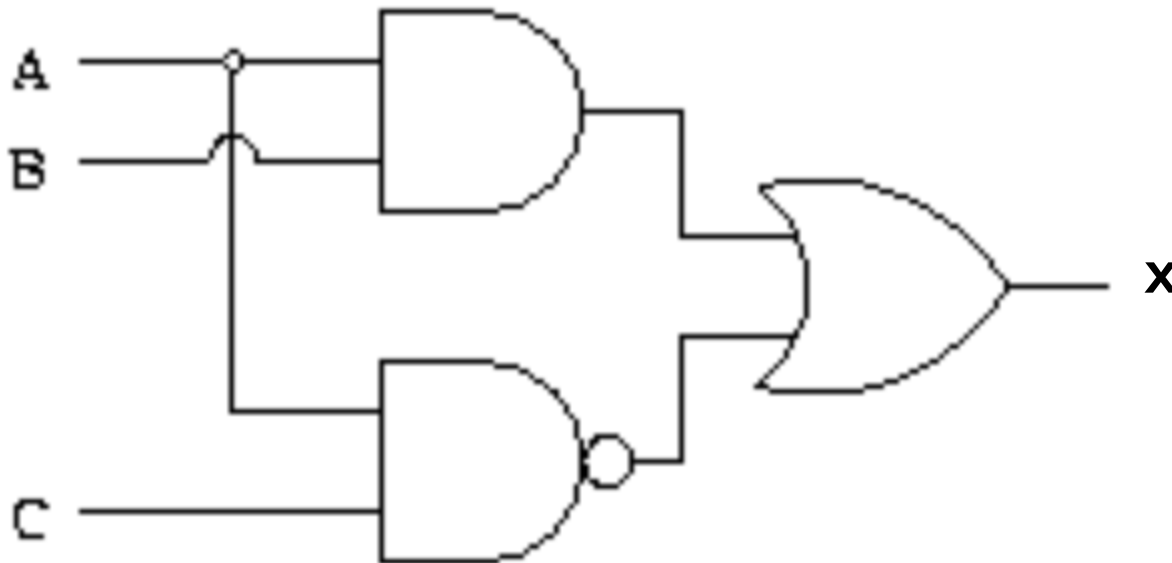
- Draw a logic circuit for $(A + B)C$

Example 3

- Draw a logic circuit for $(A + B)' + C$

Example 4

- If $A = 0$, $B = 1$ and $C = 1$, what is x ?



In-class assignment...